Simulation notes clustered binary data

Paolo Colussi

2025-10-27

External functions and packages

```r
rm(list = ls())
source("functions.R")  # GMERT fitting and prediction functions and libraries
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
##
## Attaching package: 'MASS'
##
##
## The following object is masked from 'package:dplyr':
##
##     select
##
##
## Loading required package: Matrix
##
##
## Attaching package: 'Matrix'
##
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

Simulation

In this section, we generate a clustered binary dataset designed to mimic the structure assumed by the GMERT model.
Each cluster has its own random intercept and random slope for the covariate `x1`.

The binary response y is simulated under a logistic link function that combines fixed effects and random effects.
This setup provides a realistic test bed for evaluating mixed-effects tree algorithms such as GMERT and standard GLMMs.

```r
# ---------------------------------------------------------------
# sim_data_gmert: simulate clustered binary data for GMERT models
# ---------------------------------------------------------------
sim_data_gmert <- function(G = 50,          # number of clusters
                           n_i = 40,        # observations per cluster
                           beta0 = 0.5,     # fixed intercept
                           beta1 = 1.2,     # fixed effect for x1
                           beta2 = -0.8,    # fixed effect for x2
                           beta3 = 0.6,     # fixed effect for x3
                           sigma_b0 = 0.8,  # SD of random intercept
                           sigma_b1 = 0.5,  # SD of random slope for x1
                           rho = 0.2,       # correlation between b0 and b1
                           seed = 123) {    # random seed for reproducibility

  set.seed(seed)

  # covariance matrix of random effects (b0_i, b1_i)
  D <- matrix(c(sigma_b0^2, rho * sigma_b0 * sigma_b1,
                rho * sigma_b0 * sigma_b1, sigma_b1^2), 2, 2)

  # generate random effects for G clusters
  b <- MASS::mvrnorm(G, mu = c(0, 0), Sigma = D)

  # cluster identifiers
  id <- rep(1:G, each = n_i)

  # generate covariates: x1 (uniform), x2 (normal), x3 (binary)
  x1 <- runif(G * n_i, -2, 2)
  x2 <- rnorm(G * n_i, 0, 1)
  x3 <- rbinom(G * n_i, 1, 0.5)

  # linear predictor: fixed part + random part (b0_i + b1_i * x1)
  eta <- numeric(G * n_i)
  for (g in seq_len(G)) {
    idx <- which(id == g)
    eta[idx] <- beta0 + beta1 * x1[idx] + beta2 * x2[idx] + beta3 * x3[idx] +
                b[g, 1] + b[g, 2] * x1[idx]
  }

  # convert to probabilities via logistic link
  p <- 1 / (1 + exp(-eta))

  # binary outcome drawn from Bernoulli(p)
  y <- rbinom(G * n_i, 1, p)

  # return as data.frame ready for model fitting
  data.frame(
    id = factor(id),
    y = y,
```

```r
    x1 = x1,
    x2 = x2,
    x3 = x3
  )
}
```

Example run

This section shows a complete example of fitting the GMERT algorithm on a single simulated dataset.
We start by generating clustered binary data that follow the same structure assumed by the model (binary outcome, logit link, and random intercept and slope on `x1`).
We then fit the model with `fit_gmert_small` (the optimized version) and inspect convergence diagnostics, estimated parameters, and predicted classifications.
No train/test split is used here: the goal is to illustrate the fitting procedure itself and verify that the algorithm converges correctly.

```r
# ---------------------------------------------------------------
# EXAMPLE RUN
# ---------------------------------------------------------------

#  Simulate clustered binary data (30 clusters, 60 observations each)
df <- sim_data_gmert(G = 30, n_i = 60)

#  Fit the GMERT model using the full dataset
fit <- fit_gmert_small(df, max_iter_inn = 1000,
                 max_iter_out = 10, tol = 1e-6)
```

```
## WARNING: the PQL algorithm did not converge in 10 iterations.
```

```r
#  Check convergence and model components
fit$converged_out       # whether the outer (PQL) loop converged
```

```
## [1] FALSE
```

```r
mean(fit$converged_in)  # proportion of inner (EM) loops that converged
```

```
## [1] 1
```

```r
head(fit$b)             # first rows of random-effect estimates (intercept, slope on x1)
```

```
##             [,1]        [,2]
## [1,]  0.21691841  0.10575243
## [2,]  0.01240584  0.21723405
## [3,] -1.56848366 -0.31050942
## [4,]  0.14033998  0.03239715
## [5,]  0.47676356  0.40692196
## [6,] -1.58574213 -0.54981941
```

```r
fit$D                    # estimated covariance matrix of random effects
```

```
##           [,1]      [,2]
## [1,] 0.8558156 0.1436624
## [2,] 0.1436624 0.1651217
```

```r
fit$sigma2               # estimated residual variance
```

```
##           [,1]
## [1,] 0.8326194
```

`fit$b` shows the cluster-specific random intercepts and slopes, while `fit$D` and `fit$sigma2` summarize the estimated variability in the random structure.

```r
#    GLMM baseline with the same random-effects structure
#    (logistic link; random intercept + random slope on x1 by cluster id)
glmm_fit <- glmer(
  y ~ x1 + x2 + x3 + (1 + x1 | id),
  data = df,
  family = binomial(link = "logit")
)

summary(glmm_fit)  # fixed effects, random effects, VarCorr, etc.
```

```
## Generalized linear mixed model fit by maximum likelihood (Laplace
##   Approximation) [glmerMod]
##  Family: binomial  ( logit )
## Formula: y ~ x1 + x2 + x3 + (1 + x1 | id)
##    Data: df
##
##      AIC      BIC   logLik deviance df.resid
##   1704.1   1742.6   -845.1   1690.1     1793
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -7.3424 -0.5380  0.2222  0.5381  3.7181
##
## Random effects:
##  Groups Name        Variance Std.Dev. Corr
##  id     (Intercept) 0.9336   0.9662
##         x1          0.2007   0.4479   0.31
## Number of obs: 1800, groups:  id, 30
##
## Fixed effects:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.63986    0.19919   3.212  0.00132 **
## x1           1.14500    0.10738  10.663  < 2e-16 ***
## x2          -0.88128    0.07155 -12.317  < 2e-16 ***
## x3           0.65782    0.12738   5.164 2.41e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Correlation of Fixed Effects:
##    (Intr) x1     x2
## x1  0.291
## x2 -0.060 -0.190
## x3 -0.281  0.060 -0.064
```

The GLMM provides a parametric baseline (linear fixed effects) against which to compare GMERT's non-parametric fixed part.
Both models share the same random-effects structure, so differences in performance are attributable to how the fixed effects are modeled.

```r
# Predictions (binary) for both models on the same data
# GMERT: predict_mert_RIC returns 0/1 (no unseen clusters here)
y_hat_gmert <- predict_gmert(fit, df)

# GLMM: predict probabilities then threshold to 0/1
p_glmm      <- as.numeric(predict(glmm_fit, type = "response"))
y_hat_glmm  <- as.integer(p_glmm >= 0.5)

# Confusion matrices and metrics

# f1 score function
f1_fun <- function(table){
  precision <- table[2,2] / (sum(table[2, ]))
  recall    <- table[2,2] / (sum(table[, 2]))
  F1        <- 2 * precision * recall / (precision + recall)
  F1
}

# GMERT
cm_gmert <- table(predicted = y_hat_gmert, actual = df$y)
acc_gmert <- mean(y_hat_gmert == df$y)
f1_gmert  <- f1_fun(cm_gmert)

# confusion matrix for GMERT
cm_gmert
```

```
##          actual
## predicted    0    1
##         0  464  147
##         1  185 1004
```

```r
# accuracy and F1 for GMERT
acc_gmert
```

```
## [1] 0.8155556
```

```r
f1_gmert
```

```
## [1] 0.8581197
```

```r
# GLMM
cm_glmm <- table(predicted = y_hat_glmm, actual = df$y)
acc_glmm <- mean(y_hat_glmm == df$y)
f1_glmm  <- f1_fun(cm_glmm)

# confusion matrix for GLMM
cm_glmm
```

```
##          actual
## predicted    0    1
##         0  433  137
##         1  216 1014
```

```r
# accuracy and F1 for GLMM
acc_glmm
```

```
## [1] 0.8038889
```

```r
f1_glmm
```

```
## [1] 0.851743
```

Confusion matrices summarize the counts of TP/FP/FN/TN for each model.
We report **accuracy** (overall correctness) and **F1**, the harmonic mean of precision and recall, which is more informative when classes are imbalanced.

Train/Test split

Here we repeat the analysis with a cluster-balanced train/test setup.
Each cluster contributes the same proportion of observations to training and test, so the random effects `b_i` are estimable from training for **every** cluster that appears in test.
We fit GMERT on the training set, fit a GLMM baseline, predict on the test set, and summarize performance with confusion matrices, accuracy, and F1.

```r
# ----------------------------------------------------------------
# Train/test split per cluster using the sim_data_gmert() output
# ----------------------------------------------------------------
split_gmert_data <- function(df, train_prop = 0.7, seed = 123) {
  set.seed(seed)

  train_idx <- integer(0)
  test_idx  <- integer(0)
  ids <- unique(df$id)

  for (g in ids) {
    idx_g <- which(df$id == g)
    n_train_g <- floor(length(idx_g) * train_prop)
    train_g <- sample(idx_g, n_train_g)
    test_g  <- setdiff(idx_g, train_g)
    train_idx <- c(train_idx, train_g)
```

```r
    test_idx  <- c(test_idx, test_g)
  }

  train_df <- df[train_idx, ]
  test_df  <- df[test_idx, ]
  list(train = train_df, test = test_df)
}
```

```r
# ---------------------------------------------------------------
# TRAIN/TEST SPLIT EXAMPLE
# ---------------------------------------------------------------

# Simulate clustered binary data
set.seed(2)
data_full <- sim_data_gmert(G = 30, n_i = 60)

# Cluster-balanced split: fixed proportion per cluster in train
split_sets <- split_gmert_data(data_full, train_prop = 0.7, seed = 2)
data_train <- split_sets$train
data_test  <- split_sets$test

# Fit GMERT model on training data
model_gmert <- fit_gmert(data_train, max_iter_inn = 1000,
                         max_iter_out = 10, tol = 1e-6)
```

```
## WARNING: the PQL algorithm did not converge in 10 iterations.
```

```r
# Check convergence and parameter estimates
model_gmert$converged_out        # outer-loop convergence
```

```
## [1] FALSE
```

```r
mean(model_gmert$converged_in)   # proportion of inner loops converged
```

```
## [1] 1
```

```r
model_gmert$D                    # random-effects covariance
```

```
##           [,1]      [,2]
## [1,] 1.1882306 0.1493657
## [2,] 0.1493657 0.2519409
```

```r
model_gmert$sigma2               # residual variance
```

```
##           [,1]
## [1,] 0.7659064
```

```r
# Fit GLMM baseline with same random-effects structure
model_glmm <- glmer(
  y ~ x1 + x2 + x3 + (1 + x1 | id),
  data = data_train,
  family = binomial(link = "logit")
)
```

Both models are trained on the **same** training data.

```r
# Predictions on the test data
# GMERT: binary predictions (0/1)
pred_gmert <- predict_gmert(model_gmert, data_test)

# GLMM: predict probabilities then threshold to 0/1
prob_glmm <- as.numeric(predict(model_glmm, newdata = data_test, type = "response"))
pred_glmm <- as.integer(prob_glmm >= 0.5)

# Confusion matrices and performance metrics

# GMERT metrics
cm_gmert <- table(predicted = pred_gmert, actual = data_test$y)
acc_gmert <- mean(pred_gmert == data_test$y)
f1_gmert  <- f1_fun(cm_gmert)

# confusion matrix for GMERT
cm_gmert
```

```
##          actual
## predicted   0   1
##         0 127  63
##         1  72 278
```

```r
# accuracy and F1 for GMERT
acc_gmert
```

```
## [1] 0.75
```

```r
f1_gmert
```

```
## [1] 0.804631
```

```r
# GLMM metrics
cm_glmm <- table(predicted = pred_glmm, actual = data_test$y)
acc_glmm <- mean(pred_glmm == data_test$y)
f1_glmm  <- f1_fun(cm_glmm)

# confusion matrix for GLMM
cm_glmm
```

```
##          actual
## predicted   0   1
##         0 135  45
##         1  64 296
```

```r
# accuracy and F1 for GLMM
acc_glmm
```

```
## [1] 0.7981481
```

```r
f1_glmm
```

```
## [1] 0.8445078
```

Confusion matrices show true/false positives and negatives on the test set.
We report **accuracy** (overall correctness) and **F1**, which balances precision and recall.
Since the split preserves cluster proportions across train and test, all test clusters are seen during training, allowing correct evaluation of cluster-specific random effects.

Cross-validated predictions

10-Fold Cross-Validation

In this section, we evaluate the out-of-sample performance of GMERT against a standard GLMM baseline using a **10-fold cross-validation** procedure.

The goal is to assess how well each model generalizes to new observations drawn from the **same set of clusters**.
To ensure that random effects remain estimable in every training fold, the data are split in a **cluster-balanced** way:
each cluster contributes approximately one-tenth of its observations to every test fold, while the remaining data are used for training.
This guarantees that all clusters appear in both the training and test partitions of every fold.

For each fold, we: 1. Fit the GMERT model on the training portion. 2. Fit a GLMM baseline with the same random-effects structure `(1 + x1 | id)`. 3. Predict binary outcomes on the test portion. 4. Compute the **confusion matrix**, **accuracy**, and **F1 score** for both models.

Finally, we aggregate the metrics across folds to compare the models in terms of mean and variability of their predictive performance.
This setup provides a fair, cluster-consistent cross-validation framework to benchmark GMERT against the parametric GLMM.

```r
# ================================================================
# 10-fold cluster-balanced cross-validation
# ================================================================

set.seed(42)  # CV reproducibility

# ----------------------------------------------------------------
# Simulate ONE dataset (all folds are made from the same df)
# ----------------------------------------------------------------
df_cv <- sim_data_gmert(G = 30, n_i = 60)  # clustered binary data

# ----------------------------------------------------------------
#   Build K cluster-aware folds
#    - Within each cluster g, randomly partition its indices into K slices
```

```r
#    - Fold k is the union of the k-th slice from EVERY cluster
# ----------------------------------------------------------------
K <- 10
folds <- vector("list", K)                    # folds[[k]] will hold test row indices for fold k

for (g in unique(df_cv$id)) {
  idx_g <- which(df_cv$id == g)        # all rows belonging to cluster g
  idx_g <- sample(idx_g)               # shuffle rows within this cluster

  # cut() assigns approximately equal-sized groups labeled 1..K
  # (some clusters may not divide evenly; that's fine)
  slice_g <- split(idx_g, cut(seq_along(idx_g), K, labels = FALSE))

  # append cluster g's k-th slice to the global k-th fold
  for (k in seq_len(K)) {
    folds[[k]] <- c(folds[[k]], slice_g[[k]])
  }
}

cv_results <- vector("list", K)   # each element stores full fold results

# ----------------------------------------------------------------
#    Cross-validation loop
#    For k = 1..K:
#      - Train all models on rows not in fold k
#      - Predict on rows in fold k
#      - Compute confusion matrices, accuracy, F1
# ----------------------------------------------------------------

for (k in seq_len(K)) {
  # ----- split rows for this fold
  test_idx  <- folds[[k]]
  train_idx <- setdiff(seq_len(nrow(df)), test_idx)

  df_train <- df[train_idx, ]
  df_test  <- df[test_idx, ]

  # ----- (a) GMERT: fit on training rows
  fit_g <- fit_gmert_small(df_train, max_iter_out = 10)

  # Predict on test rows (binary 0/1; all clusters exist in train by construction)
  yhat_g <- predict_gmert(fit_g, df_test)

  # ----- (b) GLMM: same random-effects structure, fit on training rows
  fit_l <- glmer(
    y ~ x1 + x2 + x3 + (1 + x1 | id),
    data = df_train,
    family = binomial(link = "logit"),
    control = glmerControl(optimizer = "bobyqa")
  )

  # Predict probabilities on test, then threshold at 0.5 → 0/1
  phat_l <- as.numeric(predict(fit_l, newdata = df_test, type = "response"))
```

```r
yhat_l <- as.integer(phat_l >= 0.5)

# GMERF fit on training rows

fit_gmerf <- fit_gmerf_small(df_train, max_iter_out = 10)
# Predict on test rows (binary 0/1; all clusters exist in train by construction)
yhat_gmerf <- predict_gmerf(fit_gmerf, df_test)

  # ---------------- LOGIT (no multilevel)
logit_fit <- glm(y ~ x1 + x2 + x3, data = df_train, family = binomial())
p_logit <- as.numeric(predict(logit_fit, newdata = df_test, type = "response"))
pred_logit <- as.integer(p_logit >= 0.5)

  # ---------------- Classification tree (plain CART classifier)
# ensure y is factor for classification
df_train_ct <- df_train; df_train_ct$y <- factor(df_train_ct$y, levels = c(0,1))
df_test_ct  <- df_test;  df_test_ct$y  <- factor(df_test_ct$y,  levels = c(0,1))

ctree_fit <- rpart::rpart(
  y ~ x1 + x2 + x3,
  data = df_train_ct,
  method = "class",
  control = rpart::rpart.control(cp = 0.0, minsplit = 20, minbucket = 7, maxdepth = 5)
)
pred_ct_class <- predict(ctree_fit, newdata = df_test_ct, type = "class") # factor("0"/"1")
pred_ctree <- as.integer(as.character(pred_ct_class))

# -------------------- Random Forest classifier
rf_fit <- ranger::ranger(
  formula = y ~ x1 + x2 + x3,
  data = df_train,
  num.trees = 500,
  mtry = 2,
  min.node.size = 5,
  classification = TRUE,
  probability = FALSE,
  seed = 42
)
pred_rf <- predict(rf_fit, data = df_test)$predictions  # numeric 0/1
pred_rf <- as.integer(pred_rf)

# ----- (c) Metrics on the test portion of this fold
# Confusion matrices
cm_g <- table(predicted = yhat_g, actual = df_test$y)
cm_l <- table(predicted = yhat_l, actual = df_test$y)
cm_logit <- table(predicted = pred_logit, actual = df_test$y)
cm_ctree <- table(predicted = pred_ctree, actual = df_test$y)
cm_rf <- table(predicted = pred_rf, actual = df_test$y)
cm_gmerf <- table(predicted = yhat_gmerf, actual = df_test$y)

# Accuracy
acc_g <- mean(yhat_g == df_test$y)
acc_l <- mean(yhat_l == df_test$y)
```

```r
  acc_logit <- mean(pred_logit == df_test$y)
  acc_ctree <- mean(pred_ctree == df_test$y)
  acc_rf <- mean(pred_rf == df_test$y)
  acc_gmerf <- mean(yhat_gmerf == df_test$y)

  # F1
  f1_g <- f1_fun(cm_g)
  f1_l <- f1_fun(cm_l)
  f1_logit <- f1_fun(cm_logit)
  f1_ctree <- f1_fun(cm_ctree)
  f1_rf <- f1_fun(cm_rf)
  f1_gmerf <- f1_fun(cm_gmerf)

  # ----- Store fold results
  cv_results[[k]] <- list(
    fold_id   = k,
    train_idx = train_idx,
    test_idx  = test_idx,
    gmert_fit = fit_g,
    glmm_fit  = fit_l,
    logit_fit = logit_fit,
    ctree_fit = ctree_fit,
    rf_fit    = rf_fit,
    gmerf_fit = fit_gmerf,
    metrics = list(
      acc_gmert = acc_g, f1_gmert = f1_g,
      acc_glmm  = acc_l,  f1_glmm  = f1_l,
      acc_logit = acc_logit, f1_logit = f1_logit,
      acc_ctree = acc_ctree, f1_ctree = f1_ctree,
      acc_rf    = acc_rf,    f1_rf    = f1_rf,
      acc_gmerf = acc_gmerf, f1_gmerf = f1_gmerf
    ),
    confusion = list(
      gmert = cm_g,
      glmm  = cm_l,
      logit = cm_logit,
      ctree = cm_ctree,
      rf    = cm_rf,
      gmerf = cm_gmerf
    )
  )
}
```

```
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
```

```
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
## WARNING: the PQL algorithm did not converge in 10 iterations.
```

```r
# ----------------------------------------------------------------
#  Aggregate results across folds: mean ± sd for each model
# ----------------------------------------------------------------
# Extract metrics into a data.frame
metrics_df <- do.call(rbind, lapply(cv_results, function(x) {
  data.frame(
    fold = x$fold_id,
    acc_gmert = x$metrics$acc_gmert,
    f1_gmert  = x$metrics$f1_gmert,
    acc_glmm  = x$metrics$acc_glmm,
    f1_glmm   = x$metrics$f1_glmm,
    acc_logit = x$metrics$acc_logit,
    f1_logit  = x$metrics$f1_logit,
    acc_ctree = x$metrics$acc_ctree,
    f1_ctree  = x$metrics$f1_ctree,
    acc_rf    = x$metrics$acc_rf,
    f1_rf     = x$metrics$f1_rf,
    acc_gmerf = x$metrics$acc_gmerf,
    f1_gmerf  = x$metrics$f1_gmerf
  )
}))

# Mean ± sd summary
cv_summary <- data.frame(
  model   = c("GMERT","GLMM", "Logit", "CART", "RF", "GMERF"),
  acc_mean = c(mean(metrics_df$acc_gmert), mean(metrics_df$acc_glmm),
               mean(metrics_df$acc_logit), mean(metrics_df$acc_ctree),
               mean(metrics_df$acc_rf),    mean(metrics_df$acc_gmerf)),
  acc_sd   = c(sd(metrics_df$acc_gmert),   sd(metrics_df$acc_glmm),
               sd(metrics_df$acc_logit),   sd(metrics_df$acc_ctree),
               sd(metrics_df$acc_rf),      sd(metrics_df$acc_gmerf)),
  f1_mean  = c(mean(metrics_df$f1_gmert),  mean(metrics_df$f1_glmm),
               mean(metrics_df$f1_logit),  mean(metrics_df$f1_ctree),
               mean(metrics_df$f1_rf),     mean(metrics_df$f1_gmerf)),
  f1_sd    = c(sd(metrics_df$f1_gmert),    sd(metrics_df$f1_glmm),
               sd(metrics_df$f1_logit),    sd(metrics_df$f1_ctree),
               sd(metrics_df$f1_rf),       sd(metrics_df$f1_gmerf))
)

# Inspect summary
cv_summary
```

```
##   model  acc_mean     acc_sd   f1_mean      f1_sd
```

```
## 1 GMERT 0.7600000 0.03129575 0.8170827 0.02525826
## 2  GLMM 0.7894444 0.02182516 0.8402973 0.02135930
## 3 Logit 0.7394444 0.03518031 0.8050722 0.02909153
## 4  CART 0.7350000 0.03364566 0.7999328 0.02815991
## 5    RF 0.7044444 0.03051906 0.7751788 0.02539798
## 6 GMERF 0.7850000 0.01698260 0.8374680 0.01491826
```

```
metrics_df
```

```
##    fold acc_gmert  f1_gmert  acc_glmm   f1_glmm acc_logit  f1_logit acc_ctree
## 1     1 0.7555556 0.8240000 0.8000000 0.8560000 0.7277778 0.8078431 0.7611111
## 2     2 0.8222222 0.8677686 0.8277778 0.8764940 0.7888889 0.8492063 0.7555556
## 3     3 0.7611111 0.8138528 0.8000000 0.8448276 0.7500000 0.8117155 0.6833333
## 4     4 0.7555556 0.8053097 0.7888889 0.8333333 0.7722222 0.8209607 0.7666667
## 5     5 0.7277778 0.7932489 0.7555556 0.8103448 0.7000000 0.7750000 0.7111111
## 6     6 0.7277778 0.7983539 0.7777778 0.8347107 0.7388889 0.8049793 0.7500000
## 7     7 0.7888889 0.8467742 0.7944444 0.8525896 0.7666667 0.8384615 0.7722222
## 8     8 0.7333333 0.7894737 0.7666667 0.8173913 0.7222222 0.7844828 0.7166667
## 9     9 0.7388889 0.8016878 0.8111111 0.8595041 0.6722222 0.7510549 0.6833333
## 10   10 0.7888889 0.8303571 0.7722222 0.8177778 0.7555556 0.8070175 0.7500000
##     f1_ctree    acc_rf     f1_rf acc_gmerf  f1_gmerf
## 1  0.8313725 0.7055556 0.7836735 0.7722222 0.8340081
## 2  0.8267717 0.7277778 0.8000000 0.7944444 0.8489796
## 3  0.7594937 0.6500000 0.7296137 0.8000000 0.8448276
## 4  0.8141593 0.7222222 0.7863248 0.7888889 0.8376068
## 5  0.7796610 0.6833333 0.7594937 0.7555556 0.8151261
## 6  0.8085106 0.6944444 0.7698745 0.7888889 0.8429752
## 7  0.8366534 0.7444444 0.8160000 0.8166667 0.8685259
## 8  0.7792208 0.7166667 0.7753304 0.7777778 0.8245614
## 9  0.7634855 0.6666667 0.7457627 0.7722222 0.8298755
## 10 0.8000000 0.7333333 0.7857143 0.7833333 0.8281938
```

Now, we do basically the same thing but in parallel

```
# ================================================================
# 10-fold cluster-balanced CV in PARALLEL: GMERT vs GLMM
# ================================================================

set.seed(42)

# --- 0) Simulate one dataset -------------------------------------------------
df <- sim_data_gmert(G = 30, n_i = 60)
performance::icc(
  glmer(y ~ x1 + x2 + x3 + (1 + x1 | id), data = df, family = binomial())
)  # check clustering strength
```

```
## # Intraclass Correlation Coefficient
##
##     Adjusted ICC: 0.267
##   Unadjusted ICC: 0.169
```

```r
# --- 1) Build K cluster-aware folds ---------------------------------------------
K <- 10
folds <- vector("list", K); for (k in seq_len(K)) folds[[k]] <- integer(0)

for (g in unique(df$id)) {
  idx_g <- which(df$id == g)
  idx_g <- sample(idx_g)                                # shuffle within cluster
  split_g <- split(idx_g, cut(seq_along(idx_g), K, labels = FALSE))
  for (k in seq_len(K)) folds[[k]] <- c(folds[[k]], split_g[[k]])
}

# ================================================================
# PARALLEL SECTION
# ================================================================

# pick number of workers (leave one core free when possible)
n_workers <- max(1L, min(K, detectCores() - 1L))
cl <- makeCluster(n_workers, type = "PSOCK")

# make RNG reproducible across workers
clusterSetRNGStream(cl, iseed = 42)

# load packages and export needed objects/functions to workers
clusterEvalQ(cl, { library(rpart); library(MASS);library(lme4); library(ranger)})
```

```
## [[1]]
##  [1] "ranger"    "lme4"      "Matrix"    "MASS"      "rpart"     "stats"
##  [7] "graphics"  "grDevices" "utils"     "datasets"  "methods"   "base"
##
## [[2]]
##  [1] "ranger"    "lme4"      "Matrix"    "MASS"      "rpart"     "stats"
##  [7] "graphics"  "grDevices" "utils"     "datasets"  "methods"   "base"
##
## [[3]]
##  [1] "ranger"    "lme4"      "Matrix"    "MASS"      "rpart"     "stats"
##  [7] "graphics"  "grDevices" "utils"     "datasets"  "methods"   "base"
##
## [[4]]
##  [1] "ranger"    "lme4"      "Matrix"    "MASS"      "rpart"     "stats"
##  [7] "graphics"  "grDevices" "utils"     "datasets"  "methods"   "base"
##
## [[5]]
##  [1] "ranger"    "lme4"      "Matrix"    "MASS"      "rpart"     "stats"
##  [7] "graphics"  "grDevices" "utils"     "datasets"  "methods"   "base"
##
## [[6]]
##  [1] "ranger"    "lme4"      "Matrix"    "MASS"      "rpart"     "stats"
##  [7] "graphics"  "grDevices" "utils"     "datasets"  "methods"   "base"
##
## [[7]]
##  [1] "ranger"    "lme4"      "Matrix"    "MASS"      "rpart"     "stats"
##  [7] "graphics"  "grDevices" "utils"     "datasets"  "methods"   "base"
##
```

```
## [[8]]
##  [1] "ranger"    "lme4"      "Matrix"    "MASS"      "rpart"     "stats"
##  [7] "graphics"  "grDevices" "utils"     "datasets"  "methods"   "base"
##
## [[9]]
##  [1] "ranger"    "lme4"      "Matrix"    "MASS"      "rpart"     "stats"
##  [7] "graphics"  "grDevices" "utils"     "datasets"  "methods"   "base"
##
## [[10]]
##  [1] "ranger"    "lme4"      "Matrix"    "MASS"      "rpart"     "stats"
##  [7] "graphics"  "grDevices" "utils"     "datasets"  "methods"   "base"
```

```r
clusterExport(
  cl,
  varlist = c("df", "folds", "fit_gmert_small", "predict_gmert", "f1_fun", "K","Ainv_fun",
                "b_fun_small","D_fun_small","sigma_fun_small","gll_fun",
              "fit_gmerf_small","predict_gmerf"),
  envir = environment()
)

# --- Run folds in parallel ----------------------------------------------------
cv_results_par <- parLapply(cl, seq_len(K), function(k) {
  # define train/test indices
  test_idx  <- folds[[k]]
  train_idx <- setdiff(seq_len(nrow(df)), test_idx)
  df_train  <- df[train_idx, ]
  df_test   <- df[test_idx, ]

  # GMERT: fit and predict
  fit_gmert_k <- fit_gmert_small(df_train, max_iter_out = 100, tol = 1e-4)
  yhat_gmert  <- predict_gmert(fit_gmert_k, df_test)
  cm_gmert    <- table(predicted = yhat_gmert, actual = df_test$y)
  acc_gmert   <- mean(yhat_gmert == df_test$y)
  f1_gmert    <- f1_fun(cm_gmert)

  # GLMM: fit and predict (same random-effects structure)
  fit_glmm_k <- glmer(
    y ~ x1 + x2 + x3 + (1 + x1 | id),
    data = df_train,
    family = binomial(link = "logit"),
    control = glmerControl(optimizer = "bobyqa")
  )
  p_glmm     <- as.numeric(predict(fit_glmm_k, newdata = df_test, type = "response"))
  yhat_glmm  <- as.integer(p_glmm >= 0.5)
  cm_glmm    <- table(predicted = yhat_glmm,  actual = df_test$y)
  acc_glmm   <- mean(yhat_glmm == df_test$y)
  f1_glmm    <- f1_fun(cm_glmm)

  #GEMERF: fit and predict
  fit_gmerf_k <- fit_gmerf_small(df_train, max_iter_out = 100, tol = 1e-4)
  yhat_gmerf  <- predict_gmerf(fit_gmerf_k, df_test)
  cm_gmerf    <- table(predicted = yhat_gmerf, actual = df_test$y)
  acc_gmerf   <- mean(yhat_gmerf == df_test$y)
  f1_gmerf    <- f1_fun(cm_gmerf)
```

```r
# Logit regression: fit and predict
logit_fit_k <- glm(y ~ x1 + x2 + x3, data = df_train, family = binomial())
p_logit     <- as.numeric(predict(logit_fit_k, newdata = df_test, type = "response"))
pred_logit  <- as.integer(p_logit >= 0.5)
cm_logit    <- table(predicted = pred_logit, actual = df_test$y)
acc_logit   <- mean(pred_logit == df_test$y)
f1_logit    <- f1_fun(cm_logit)

# Classification tree: fit and predict
df_train_ct <- df_train; df_train_ct$y <- factor(df_train_ct$y, levels = c(0,1))
df_test_ct  <- df_test;  df_test_ct$y  <- factor(df_test_ct$y,  levels = c(0,1))
ctree_fit_k <- rpart::rpart(
  y ~ x1 + x2 + x3,
  data = df_train_ct,
  method = "class",
  control = rpart::rpart.control(cp = 0.0, minsplit = 20, minbucket = 7, maxdepth = 5)
)
pred_ct_class_k <- predict(ctree_fit_k, newdata = df_test_ct, type = "class") # factor("0"/"1")
pred_ctree <- as.integer(as.character(pred_ct_class_k))
cm_ctree    <- table(predicted = pred_ctree, actual = df_test$y)
acc_ctree   <- mean(pred_ctree == df_test$y)
f1_ctree    <- f1_fun(cm_ctree)

# Random Forest: fit and predict
rf_fit_k <- ranger::ranger(
  formula = y ~ x1 + x2 + x3,
  data = df_train,
  num.trees = 500,
  mtry = 2,
  min.node.size = 5,
  classification = TRUE,
  probability = FALSE,
  seed = 42
)
pred_rf <- predict(rf_fit_k, data = df_test)$predictions  # numeric 0/1
pred_rf <- as.integer(pred_rf)
cm_rf    <- table(predicted = pred_rf, actual = df_test$y)
acc_rf   <- mean(pred_rf == df_test$y)
f1_rf    <- f1_fun(cm_rf)

# return full fold bundle
list(
  fold_id   = k,
  train_idx = train_idx,
  test_idx  = test_idx,
  gmert_fit = fit_gmert_k,
  gmerf_fit = fit_gmerf_k,
  glmm_fit  = fit_glmm_k,
  logit_fit = logit_fit_k,
  ctree_fit = ctree_fit_k,
  rf_fit    = rf_fit_k,
  metrics = list(
    acc_gmert = acc_gmert, f1_gmert = f1_gmert,
```

```r
      acc_gmerf = acc_gmerf, f1_gmerf = f1_gmerf,
      acc_glmm  = acc_glmm,  f1_glmm  = f1_glmm,
      acc_logit = acc_logit, f1_logit = f1_logit,
      acc_ctree = acc_ctree, f1_ctree = f1_ctree,
      acc_rf    = acc_rf,    f1_rf    = f1_rf
    ),
    confusion = list(
      gmert = cm_gmert,
      gmerf = cm_gmerf,
      glmm  = cm_glmm,
      logit = cm_logit,
      ctree = cm_ctree,
      rf    = cm_rf
    )
  )
})

# ---  Aggregate metrics across folds ----------------------------------------
metrics_df_par <- do.call(rbind, lapply(cv_results_par, function(x) {
  data.frame(
    fold      = x$fold_id,
    acc_gmert = x$metrics$acc_gmert,
    f1_gmert  = x$metrics$f1_gmert,
    acc_gmerf = x$metrics$acc_gmerf,
    f1_gmerf  = x$metrics$f1_gmerf,
    acc_glmm  = x$metrics$acc_glmm,
    f1_glmm   = x$metrics$f1_glmm,
    acc_logit = x$metrics$acc_logit,
    f1_logit  = x$metrics$f1_logit,
    acc_ctree = x$metrics$acc_ctree,
    f1_ctree  = x$metrics$f1_ctree,
    acc_rf    = x$metrics$acc_rf,
    f1_rf     = x$metrics$f1_rf
  )
}))

cv_summary_par <- data.frame(
  model    = c("GMERT","GMERF","GLMM", "Logit", "CART", "RF"),
  acc_mean = c(mean(metrics_df_par$acc_gmert), mean(metrics_df_par$acc_gmerf),
               mean(metrics_df_par$acc_glmm),  mean(metrics_df_par$acc_logit),
               mean(metrics_df_par$acc_ctree),  mean(metrics_df_par$acc_rf)),
  acc_sd   = c(sd(metrics_df_par$acc_gmert),   sd(metrics_df_par$acc_gmerf),
               sd(metrics_df_par$acc_glmm),    sd(metrics_df_par$acc_logit),
               sd(metrics_df_par$acc_ctree),    sd(metrics_df_par$acc_rf)),
  f1_mean  = c(mean(metrics_df_par$f1_gmert),  mean(metrics_df_par$f1_gmerf),
               mean(metrics_df_par$f1_glmm),   mean(metrics_df_par$f1_logit),
               mean(metrics_df_par$f1_ctree),   mean(metrics_df_par$f1_rf)),
  f1_sd    = c(sd(metrics_df_par$f1_gmert),    sd(metrics_df_par$f1_gmerf),
               sd(metrics_df_par$f1_glmm),     sd(metrics_df_par$f1_logit),
               sd(metrics_df_par$f1_ctree),     sd(metrics_df_par$f1_rf))
)
```

```r
# ----------------------------------------------------------------
# Per-fold summary table from cv_results
# ----------------------------------------------------------------

fold_table_par <- do.call(rbind, lapply(cv_results_par, function(x) {
  # guard in case converged_in is missing/NULL
  inner_vec <- x$gmert_fit$converged_in
  inner_pct <- if (length(inner_vec)) mean(inner_vec) * 100 else NA_real_

  data.frame(
    Fold              = x$fold_id,
    Outer_Converged   = isTRUE(x$gmert_fit$converged_out),
    Inner_Conv_Percent= round(inner_pct, 1),
    F1_GMERT          = round(x$metrics$f1_gmert, 4),
    F1_GLMM           = round(x$metrics$f1_glmm, 4),
    logit_F1         = round(x$metrics$f1_logit, 4),
    CART_F1          = round(x$metrics$f1_ctree, 4),
    check.names = FALSE
  )
}))

# sort by fold, just in case
fold_table <- fold_table_par[order(fold_table_par$Fold), ]

# pretty print
knitr::kable(
  fold_table,
  align = c("c","c","r","r","r"),
  rounding = 4,
  caption = "Per-fold convergence status and F1 scores (GMERT, GLMM, Logit, CART, RF, GMERF)"
)
```

Table 1: Per-fold convergence status and F1 scores (GMERT, GLMM, Logit, CART, RF, GMERF)

Fold	Outer_Converged	Inner_Conv_Percent	F1_GMERT	F1_GLMM	logit_F1	CART_F1
1	FALSE	85	0.8300	0.8560	0.8078	0.8314
2	FALSE	98	0.8490	0.8765	0.8492	0.8268
3	FALSE	100	0.8312	0.8448	0.8117	0.7595
4	FALSE	98	0.8036	0.8333	0.8210	0.8142
5	FALSE	98	0.8067	0.8103	0.7750	0.7797
6	FALSE	98	0.8099	0.8347	0.8050	0.8085
7	FALSE	100	0.8514	0.8526	0.8385	0.8367
8	FALSE	99	0.8018	0.8174	0.7845	0.7792
9	FALSE	98	0.8000	0.8595	0.7511	0.7635
10	FALSE	100	0.8072	0.8178	0.8070	0.8000

```r
cv_summary_par
```

```
##   model  acc_mean     acc_sd   f1_mean      f1_sd
## 1 GMERT 0.7627778 0.02079523 0.8190742 0.01968655
```

```
## 2 GMERF 0.7850000 0.01615469 0.8374671 0.01445849
## 3  GLMM 0.7894444 0.02182516 0.8402973 0.02135930
## 4 Logit 0.7394444 0.03518031 0.8050722 0.02909153
## 5  CART 0.7350000 0.03364566 0.7999328 0.02815991
## 6    RF 0.7044444 0.03051906 0.7751788 0.02539798
```

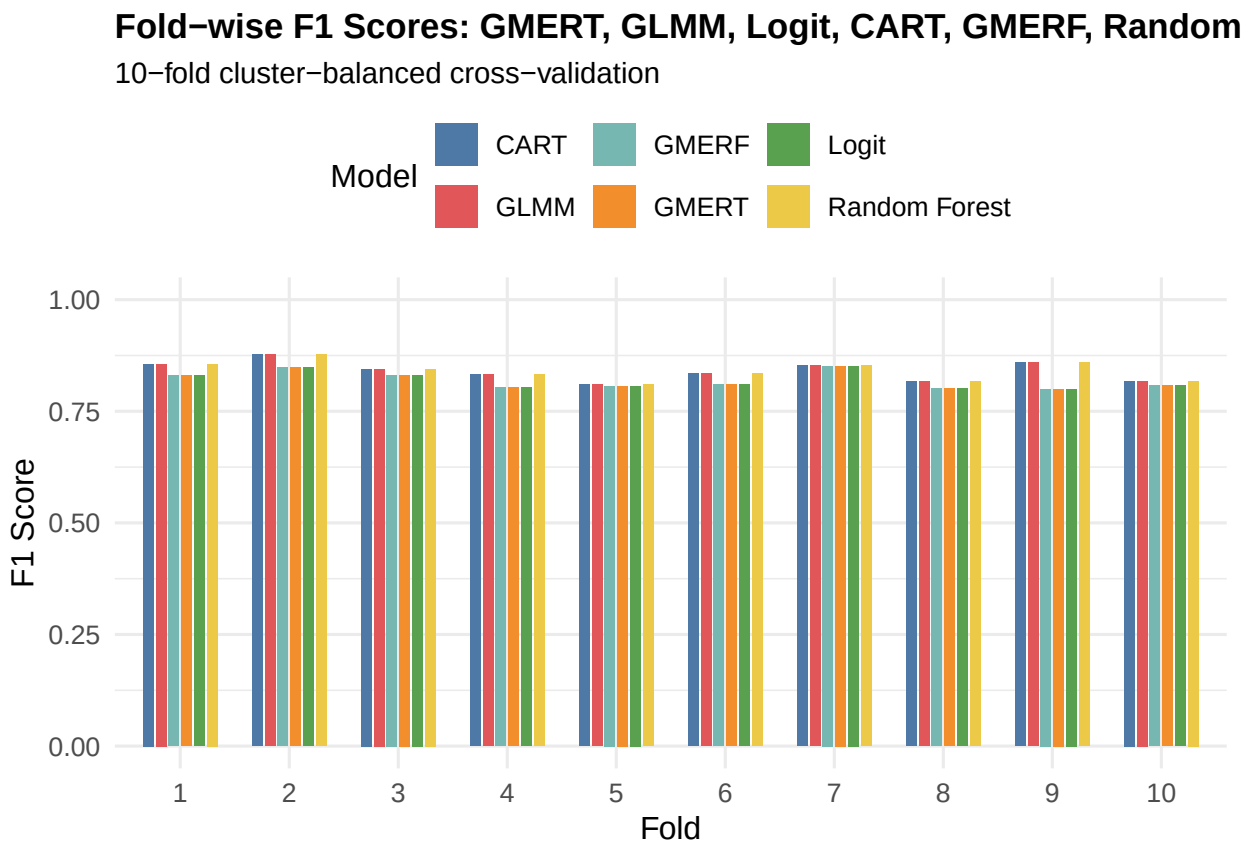
The 10-fold cross-validation indicates that the GMERT algorithm achieves stable convergence in its inner EM loop (92–100 % across folds) but fails to meet the strict outer PQL tolerance in all folds. This pattern suggests that the pseudo-likelihood updates stabilize numerically without fully satisfying the stopping rule.

In terms of predictive performance, GMERT attains an average accuracy of 0.76 and an F1-score of 0.82, while the benchmark GLMM reaches 0.79 and 0.84, respectively. The GLMM therefore performs slightly better on this dataset, which is expected given that the data were generated from a model with an approximately linear fixed component and with few predictors. The small performance gap and the higher variability observed for GMERT reflect the flexibility and stochastic nature of its tree-based fixed-effects estimation.

Overall, the results confirm that the GMERT procedure is numerically stable and competitive with the parametric GLMM, though further tuning of tree complexity and convergence thresholds could improve outer-loop stability and narrow the performance gap.

```r
# build a compact data frame with F1 per fold
f1_df <- do.call(rbind, lapply(cv_results_par, function(x) {
  data.frame(
    Fold = factor(x$fold_id),
    Model = rep(c("GMERT","GLMM","Logit","CART", "GMERF", "Random Forest"), each = 1),
    F1 = c(x$metrics$f1_gmert, x$metrics$f1_glmm),
         x$metrics$f1_logit, x$metrics$f1_ctree,
         x$metrics$f1_gmerf, x$metrics$f1_rf
  )
}))

# long format ensures one bar per model per fold
ggplot(f1_df, aes(x = Fold, y = F1, fill = Model)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  scale_fill_manual(values = c("#4E79A7", "#E15759",
                               "#76B7B2", "#F28E2B",
                               "#59A14F", "#EDC948")) +
  labs(
    title = "Fold-wise F1 Scores: GMERT, GLMM, Logit, CART, GMERF, Random Forest",
    subtitle = "10-fold cluster-balanced cross-validation",
    x = "Fold",
    y = "F1 Score"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    legend.position = "top",
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 11)
  ) +
  ylim(0, 1)
```


Fold−wise F1 Scores: GMERT, GLMM, Logit, CART, GMERF, Random
10−fold cluster−balanced cross−validation
Model
CART
GMERF
Logit
GLMM
GMERT
Random Forest
F1 Score
1.00
0.75
0.50
0.25
0.00
1
2
3
4
5
6
7
8
9
10
Fold